BSc IT FINAL YEAR ENGINEERING PROJECT | VIVA & INDUSTRY DEFENSE GUIDE

INTENTLOCK: AI Agent Security Middleware & Developer SDK

Author: Divy Bhanushali • Status: Production Ready (88/88 Automated Tests Passed • 15/15 Threat Vectors Blocked)

1. Executive Summary & 30-Second Elevator Pitch

"Autonomous AI agents are being connected directly to enterprise file systems, databases, emails, and payment APIs. However, Large Language Models are fundamentally non-deterministic and vulnerable to prompt injection attacks.

Today's industry standard is using 'system prompts' or a second LLM to police the first LLM. That is like asking one thief to watch another thief.

***INTENTLOCK** is an out-of-band, code-level Security Middleware and Python Developer SDK. It sits at the execution boundary between the AI agent and real-world tools. Every tool call is intercepted and deterministically evaluated against hard-coded policy rules, single-use cryptographic approvals, and an append-only audit trail before any code can execute. **We don't trust the AI to secure the AI. We enforce Zero Trust in code.**"*

2. The Problem vs. IntentLock Solution

Industry's Flawed Approach	IntentLock Zero-Trust Middleware
System Prompt Instructions ('Please don't delete files')	Deterministic Code Boundary: Execution fails closed unless hard-coded policy returns ALLOW.
Second LLM as Guardrail (Probabilistic, slow, bypassable)	Zero LLM Trust: LLM output is untrusted input; hard rules decide permissions.
Errors cause agent to hallucinate or bypass checks	Fail-Closed Architecture: Unknown tools, bad schema, or crashes immediately default to DENY.
Sensitive actions run automatically without oversight	Cryptographic Approvals: High-risk actions require single-use HMAC-SHA256 human tokens.
No tamper-evident record of actions	Immutable Audit Log: Every call logged with SHA-256 hashes and secret redaction.

3. The 9-Step Deterministic Policy Pipeline

- **1. Tool Registration Check:** Unregistered tools rejected immediately (Fail-Closed).
- **2. Policy Binding Verification:** Ensures agent is bound to an active security policy.
- **3. Explicit Deny Rules:** High-precedence explicit blacklist evaluation (e.g. drop database).
- **4. Explicit Allow Rules:** Checks whitelist for pre-approved safe actions.
- **5. Human Approval Requirement:** Flags sensitive actions for single-use cryptographic token authorization.
- **6. Sensitivity Ceiling:** Rejects actions exceeding numerical sensitivity limits.
- **7. Heuristic Risk Engine:** Composite risk scoring based on payload and context (rejects if > 0.8).
- **8. Recipient Domain Whitelist:** Wildcard glob matching prevents data exfiltration (e.g. *@company.com).
- **9. Resource Path Glob Matching:** Prevents path traversal and directory escape attacks (e.g. /data/inbox/*).

4. The 15 Threat Vectors Defeated (test_threat_model_pentest.py)

Threat ID & Vector	Attack Behavior / Payload	Defense Outcome
T-01: Prompt Injection	SQL injection in prompt (DROP TABLE users;--)	BLOCKED (Deterministic Policy)
T-02: Shadow Tools	Undeclared tool invocation os_system	BLOCKED (ToolNotRegisteredError)

T-03: Path Traversal	Relative path escape (../../etc/shadow)	BLOCKED (Glob Boundary Check)
T-04: Data Exfiltration	Email to hacker@evil-drop.net	BLOCKED (Domain Whitelist)
T-05: Unauthorized Payments	$50,000 wire transfer without token	BLOCKED (Escalates to Approval)
T-06: Approval Token Replay	Reusing single-use token for 2nd transfer	BLOCKED (ApprovalReplayError)
T-07: Rate Exhaustion DoS	Burst of high-frequency tool invocations	BLOCKED (Rate Limiter)
T-08: Credential Leakage	Raw API keys (sk-live-...) in parameters	REDACTED ([REDACTED] in logs)
T-09: Parameter Injection	String in integer price parameter	BLOCKED (ArgumentValidationError)
T-10: Scope Escalation	Accessing /reports/q4/ from /reports/q1/*	BLOCKED (Wildcard Glob Violation)
T-11: Gateway Bypass	Invoking tool directly around gateway	PREVENTED (Protected boundary)
T-12: Nonce Substitution	Forging token with altered action hash	BLOCKED (HMAC mismatch)
T-13: Expired Token Use	Submitting token after TTL expiry	BLOCKED (ApprovalExpiredError)
T-14: Fail-Closed on Crash	Tool raises unexpected runtime crash	FAIL-CLOSED (Action DENIED)
T-15: MCP Protocol Attack	Unauthorized JSON-RPC method to MCP proxy	BLOCKED (Standard -32601 Error)

5. Step-by-Step Live Demonstration Script

Step 1: Terminal Email Agent Demo (python examples/email_agent/demo.py)
Demonstrates ALLOW on inbox reads, DENY on path traversal (../../etc/shadow), DENY on exfiltration emails, and REQUIRE_APPROVAL for $450 refunds.

Step 2: Automated Pentest Suite (pytest sdk/python/tests/test_threat_model_pentest.py -v)
Executes 15 synthetic cyber attacks against the gateway in 0.42 seconds with 100% defense rate.

Step 3: Security Operations Center Dashboard (http://localhost:3000)
Real-time telemetry metrics, 15/15 Threat Matrix status strip, 1-click Approval Queue, and live Intercept Playground.

6. Top Viva Examination Questions & Winning Answers

Q1: Why not just instruct the LLM in its system prompt to be secure?

Answer: System prompts share the context window with untrusted user inputs. Attackers exploit this via indirect prompt injection and jailbreaks. IntentLock enforces security out-of-band in deterministic Python code where the LLM has zero authority.

Q2: How does IntentLock prevent token replay attacks?

Answer: Approval tokens are HMAC-SHA256 digests bound to the action hash, a unique nonce, and a 5-minute TTL. Once submitted, the token is permanently invalidated in memory and database; replay attempts trigger ApprovalReplayError.

Q3: What is the 'Fail-Closed' guarantee?

Answer: Any unknown tool, invalid parameter, or unexpected crash immediately defaults to DENY. Tools only run upon an explicit, verified ALLOW.

Q4: How does IntentLock integrate with modern agent frameworks?

Answer: Provides drop-in wrappers: SecureTool for LangChain/LangGraph, and MCPSecurityProxy for the Model Context Protocol (MCP).